

Motion Planning with PNO

Documentation

Samyak Soni

April 11, 2026

Contents

1	Problem Statement	2
1.1	Objective	2
1.2	Inputs	2
1.3	Outputs	2
1.4	Planning Extension	2
2	Mathematical Formulation	2
2.1	The Eikonal Partial Differential Equation	2
2.2	Interpretation	3
3	System Pipeline	3
4	Model Architectures	3
4.1	Phase 1: SDF-FNO (Geometry to Signed Distance Field)	3
4.1.1	Motivation	3
4.1.2	Architecture Specification	3
4.2	Phase 2: Planning Neural Operator (PNO)	4
4.2.1	Core Innovation	4
4.2.2	Architecture Specification	4
4.2.3	Input Construction	4
4.2.4	Smoothed Indicator Function (SIFN) Mask	4
4.2.5	DAFNO Blocks (Domain-Agnostic Fourier Neural Operator Blocks)	4
4.2.6	DeepNorm Projection Head	5
5	Experiments and Results	5
6	Conclusion	5

1 Problem Statement

1.1 Objective

Learn to compute a value function $V(x, g)$ from any 2D obstacle environment and goal position (g). The value function provides the minimum cost-to-goal from every spatial location x .

1.2 Inputs

- **Binary Occupancy Map:** $m(x) \in \{0, 1\}$ where 1 indicates free space, 0 indicates obstacles.
- **Goal Position:** $g = (g_x, g_y)$ as discrete pixel coordinates.

1.3 Outputs

- **Value Function Field:** $V(x, g)$ where each pixel holds the shortest distance/cost to reach goal g from that location.
- **Implicit Path:** The actual path is recovered via gradient descent on V : $\mathbf{p}(t+1) = \mathbf{p}(t) - \eta \nabla V(\mathbf{p}(t), g)$.

1.4 Planning Extension

The learned value function is designed to serve as an ϵ -consistent heuristic for A* search. This guarantees:

- 100% success rates (no goal-unreachability errors).
- Minimal node expansions via admissible heuristic guidance.
- Optimality with ϵ -bounded cost approximation.

2 Mathematical Formulation

2.1 The Eikonal Partial Differential Equation

The value function satisfies the Eikonal PDE:

$$\|\nabla V(x, g)\| = c(x), \quad V(g, g) = 0$$

where:

- $\nabla V(x, g)$ is the spatial gradient of the value function.
- $c(x) = m(x)$ is the local traversal cost (1 in free space, ∞ in obstacles).
- The boundary condition $V(g, g) = 0$ anchors the value at the goal.

2.2 Interpretation

- The magnitude of the gradient directly encodes the cost per unit distance.
- In free space, $|\nabla V| = 1$ implies Euclidean distance.
- On or near obstacles, the gradient is undefined (non-smooth boundary).
- Shortest paths are found via ∇V : steepest descent curves from any (x) to g .

3 System Pipeline

The overall approach:

1. **Input:** Binary occupancy map m and goal g .
2. **SDF Computation:** Convert binary map to Signed Distance Field (via FNO-based SDF model).
3. **Planning Inference:** Use PNO to map (map, SDF, goal) to value function V .
4. **Path Recovery:** Gradient descent on V traces shortest path; A* integration uses V as heuristic.

4 Model Architectures

4.1 Phase 1: SDF-FNO (Geometry to Signed Distance Field)

4.1.1 Motivation

Binary occupancy maps are non-differentiable at obstacle boundaries. To enable smooth gradient-based operations, we first convert the binary map to a continuous Signed Distance Field (SDF) using a Fourier Neural Operator (FNO).

4.1.2 Architecture Specification

The FNO2dSDF model processes single-channel input (binary map) to single-channel output (SDF):

Layer Structure:

- **Input Lift:** Linear projection from 1 channel $\rightarrow$ 32 channels (via `fc0`).
- **Spectral Blocks** ($\times 4$): Each block contains:
 - **SpectralConv2d:** Operates in Fourier space; learns complex weights for the low `modes_1` $\times$ `modes_2` frequencies.
 - **Conv2d(1x1):** Spatial mixing layer; refines feature interactions locally.
 - **Activation:** GELU.
- **Output Head:** Sequential MLP ($32 \rightarrow 128 \rightarrow 1$) with GELU; outputs single-channel SDF.

4.2 Phase 2: Planning Neural Operator (PNO)

4.2.1 Core Innovation

The PNO learns the Eikonal solution directly by jointly conditioning on three inputs: the raw binary map, the continuous SDF, and the goal location. The PNO employs domain-awareness via the Smoothed Indicator Function (SIFN) mask to robustly handle obstacle boundaries.

4.2.2 Architecture Specification

4.2.3 Input Construction

The PNO concatenates three channels:

$$\mathbf{x}_{\text{in}} = [m(x), \text{SDF}(x), \mathbf{g}(x)] \in \mathbb{R}^{B \times 3 \times H \times W}$$

where:

- $m(x)$: Binary occupancy map.
- $\text{SDF}(x)$: Pre-computed signed distance field from Phase 1.
- $\mathbf{g}(x)$: Goal channel; a sparse tensor with -1 at goal coordinates, 0 elsewhere.

4.2.4 Smoothed Indicator Function (SIFN) Mask

Before processing, a domain-aware mask is constructed using a FNO model:

$$\chi(x) = \sigma(\text{FNO}_{\text{mask}}(m, \text{SDF}))$$

where σ is a sigmoid activation ensuring $\chi \in (0, 1)$.

Purpose:

- Smooth transition from obstacles ($\chi \approx 0$) to free space ($\chi \approx 1$).
- Allows spectral layers to learn localized boundary conditions.
- Prevents spurious oscillations near walls via soft masking.

4.2.5 DAFNO Blocks (Domain-Agnostic Fourier Neural Operator Blocks)

The core processing pipeline uses 4 stacked DAFNO blocks. Each block:

- **Spectral Layer (DAFNOSpectralConv2d):**
 - Computes 2D FFT of the masked feature: $\tilde{v} = \mathcal{F}(v \odot \chi)$.
 - Applies learnable complex weights to low-frequency modes: $\tilde{v}_{\text{out}} = W \cdot \tilde{v}_{\text{in}}$.
 - Inverses with 2D iFFT: $v_{\text{out}} = \mathcal{F}^{-1}(\tilde{v}_{\text{out}})$.
- **Local Convolution:** 1×1 Conv2d; enables point-wise non-linear feature mixing.

- **Residual Connection & Activation:**

$$v_{\text{out}} = \text{GELU}(v_{\text{spec}} + v_{\text{local}})$$

4.2.6 DeepNorm Projection Head

The final layer enforces the Triangle Inequality constraint (value function consistency):

$$V(x, g) \leq V(x, x') + V(x', g) \quad \forall x, x', g$$

Implementation:

- Computes feature differences relative to the goal: $\Delta v = |v - v_{\text{goal}}|$.
- Projects through a 2-layer MLP with **positive weights only** (enforced via **softplus**):

$$V(x, g) = \sigma_2(W_2^+ \cdot \text{GELU}(W_1^+ \cdot \Delta v + b_1) + b_2)$$

where $W_i^+ = \text{softplus}(W_i)$.

- Positivity of weights ensures monotonic increase in cost with spatial distance, preserving optimality.